

Arquitetura de **Big Data** para uma Frota Autônoma Conectada

Solução de dados resiliente, escalável e em tempo real que gera **valor mensurável** para o consórcio de locadoras — fundamentada na literatura de VLDB, Data Warehouse e Big Data.

GRUPO • COMPONENTES (NOME COMPLETO + DRE)

Izabela Lima da Silva

DRE 124156557

Caio Meirelles

DRE 122071557

Estrutura da apresentação

01	Contexto e problema do negócio	COMUM	02	Valor para o cliente	COMUM
03	Objetivos e requisitos	COMUM	04	Metodologia do trabalho	COMUM
05	Fundamentos teóricos	COMUM	06	Do DW ao Big Data (+ extensão)	COMUM
07	Arquitetura de referência	COMUM	08	Camada de borda (Edge)	PARTE A
09	Ingestão — Kafka	PARTE A	10	Conectividade e ordenação	PARTE A
11	Lambda vs. Kappa	PARTE A	12	Lakehouse — Delta Lake	PARTE A
13	Processamento — Spark/Flink	PARTE A	14	Persistência poliglota (NoSQL)	PARTE B
15	Dashboard (Streamlit)	PARTE B	16	Cobrança e emergências	PARTE B
17	Concierge de IA por voz (PLN/RAG)	PARTE B	18	Casos de uso reais + rastreabilidade	COMUM
19	Custos, alternativas, referências e conclusões	COMUM			

Da locação manual à frota autônoma conectada

Um consórcio de seis locadoras automatiza a operação: pelo aplicativo, o cliente reserva o veículo e indica onde recebê-lo; o veículo **sai sozinho do pátio, navega no trânsito e estaciona no ponto indicado**; ao fim, retorna para higienização e manutenção. Cada veículo é um **sensor sobre rodas** que gera fluxo contínuo de dados.

FONTE DE DADOS

Veículo instrumentado

Sensores internos (motor, chassi) e externos (telemetria inercial, câmeras 360°, impacto) + computador de bordo (Edge).

ESCALA

Frota simultânea

Milhares de veículos transmitindo ao mesmo tempo, sob banda móvel cara e finita.

AMBIÇÃO

Autonomia e IA

Reserva sem balcão, cobrança dinâmica pós-uso, resposta a sinistros e concierge por voz.

PROBLEMA CENTRAL

O Data Warehouse sozinho não sustenta o novo modelo: é preciso uma solução de **Big Data** com ingestão massiva, processamento em tempo real e persistência poliglota.

O que o consórcio ganha com a solução

UTILIZAÇÃO DA FROTA

Menos veículos ociosos

Redistribuição inteligente (Markov + rotas em tempo real) antecipa a demanda por pátio e maximiza o giro dos ativos.

↑ taxa de utilização

RECEITA

Cobrança justa e dinâmica

Acréscimos por km, consumo e infrações — receita que hoje se perde — com transparência que reduz contestações.

↑ receita / locação

CUSTO OPERACIONAL

Sem balcão, sem logística de entrega

Reserva pelo app e veículo autônomo cortam o custo de atendimento e de deslocar equipes para entregar/buscar.

↓ custo por locação

SINISTROS

Resposta e regulação mais rápidas

Deteção na borda + dossiê automático aceleram o socorro e o processo com seguradora/órgãos.

↓ tempo de resposta

DISPONIBILIDADE

Manutenção preditiva

Antecipar panes a partir da telemetria reduz o veículo parado e a quebra em operação.

↓ downtime

EXPERIÊNCIA

Concierge por IA

Roteiro personalizado por voz diferencia o serviço e fideliza o cliente premium.

↑ satisfação / NPS

FIO CONDUTOR

Cada decisão técnica a seguir existe para mover um destes **indicadores de negócio** — a arquitetura é meio, o valor é o fim.

O que a solução precisa entregar

REQUISITOS FUNCIONAIS

- **Acompanhamento** (dashboard): frota, emergências e financeiro
- **Reserva** mais objetiva, simples e eficiente
- **Cobrança automática** pós-devolução, com ajustes por uso
- **Apoio a emergências** e dossiê para regulação
- *Extra: concierge de viagem por IA* (voz)

REQUISITOS NÃO FUNCIONAIS (5 DESAFIOS)

- **Banda finita** — inviável trafegar vídeo bruto contínuo
- **Alta concorrência** — sem perda de pacotes nem gargalo
- **Ordenação estrita** por veículo e timestamp
- **Resiliência** — tolerância a falhas e reprocessamento
- **Consistência ACID** entre lote e streaming

Estes requisitos não funcionais orientam todas as decisões de arquitetura apresentadas a seguir.

Como a proposta foi construída

1

Levantamento de requisitos

Requisitos funcionais e não funcionais extraídos do contexto e do enunciado.

2

Revisão bibliográfica

Leitura de **21 referências primárias** (papers seminais) mapeadas às 4 partes da ementa.

3

Decisão fundamentada

Cada escolha por *trade-offs* explícitos, ancorada na literatura.

4

Seleção de componentes

Tecnologia escolhida pelo padrão de acesso (persistência poliglota).

5

Prova de conceito

Sistema executável (docker-compose) validado; DDL do DW estendido testado.

6

Continuidade com o DW

Extensão do modelo dimensional da Avaliação 02 — sem retrabalho.

Critério de decisão em cada camada: adequação ao SLA · custo · resiliência · complexidade — sempre com lastro teórico citado.

Ombros de gigantes: as bases do projeto

Cada camada é ancorada na literatura fundadora de VLDB, DW e Big Data — organizada nas quatro partes da ementa.

PARTE	PRINCÍPIO FUNDADOR	REFERÊNCIA	ONDE APLICAMOS
I · Distribuído / VLDB	Falha é a norma; mover a computação para perto do dado (localidade)	Ghemawat 2003; Dean & Ghemawat 2004	Réplicas, Edge, escalonamento do Spark
I · Tempo distribuído	Ordem e consistência externa (TrueTime)	Corbett 2012	Ordenação estrita por veículo+timestamp
II · DW / OLAP / Lakehouse	OLAP separado do OLTP; operador CUBE; Lakehouse unifica	Chaudhuri & Dayal 1997; Gray 1997; Armbrust 2021	DW vira camada Gold; cubo executivo
III · Streaming	Log particionado; watermarks; exactly-once	Kreps 2011; Carbone 2015; Zaharia 2013	Kafka, Flink, Spark Streaming
III · Engenharia de IA	Geração ancorada em conhecimento recuperável (RAG)	Lewis 2020; Guo 2022	Concierge e dossiês citáveis
IV · NoSQL / CAP	CAP; BASE vs ACID; wide-column LSM	Gilbert & Lynch 2002; DeCandia 2007; Lakshman 2009	Persistência poliglota

Do Data Warehouse ao Big Data – e a extensão do modelo

AVALIAÇÃO 02 • OLTP → OLAP [CHAUDHURI & DAYAL 1997]

Esquema estrela (PostgreSQL)

- Fatos: **Reserva, Locação, Movimentação** (fact constellation)
- 5 dimensões conformadas; Pátio *role-playing*
- 4 fontes OLTP → ELT; previsão de pátios via **Markov**

AVALIAÇÃO 03 • O DW VIRA A CAMADA GOLD

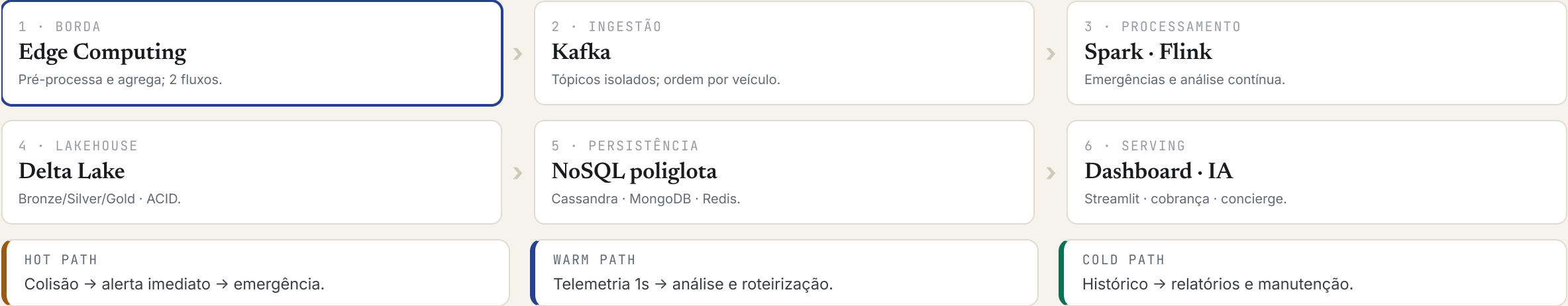
Extensão dimensional

- Novos fatos: **Telemetria, Cobrança, Sinistro, Manutenção**
- Novas dims: Tempo_Detalhe (min), Sensor (SCD2), TipoEvento, FaixaCondução
- Carregados por **ELT do Silver** (Flink/Spark) no Lakehouse

ESTRATÉGIA — LAKEHOUSE [ARMBRUST 2021]

Não substituir, e sim **envolver** o DW: o esquema estrela passa a ser a camada **Gold** sobre Delta/MinIO, alimentada pelo pipeline de streaming — evita a dicotomia lake vs. warehouse.

Pipeline fim a fim: ingestão → processamento → armazenamento → visualização



Filtrar na borda: só o relevante trafega pela rede

Fundamento: **localidade** — mover a computação para perto do dado poupa a banda escassa; a pré-agregação no veículo é o *combiner* do MapReduce aplicado à borda. [Dean & Ghemawat 2004]

FLUXO DE ROTINA • WARM PATH

Telemetria agregada

- Velocidade, GPS, bateria, combustível, sensores
- Agregada e enviada a cada **1 s (ou mais)**
- Binário compacto: Avro sobre MQTT

FLUXO CRÍTICO • HOT PATH

Alerta imediato de sinistro

- Detecção de colisão **na borda** (reação sub-segundo)
- Prioriza a rede e envia *payload* de emergência
- *Ring buffer*: ~30 s de dados brutos + metadados

VALOR

Corta o **maior custo recorrente** (banda/egress) e viabiliza a resposta que não pode esperar a nuvem. O vídeo bruto permanece no veículo.

Apache Kafka como log distribuído particionado

Log particionado com consumo *pull* por offset e retenção temporal — buffer durável sob banda finita, com **ordem estrita dentro da partição**. [Kreps 2011]

TÓPICOS ISOLADOS

Separar telemetria de alarmes

`telemetry.raw``alerts.critical``trips.events``billing.events`

O alarme crítico **nunca fica preso** atrás da fila de telemetria.

GARANTIAS

Ordem e resiliência

- Partição por `vehicle_id` → ordem por veículo, sem coordenação global
- Replicação + ISR → tolerância a falhas [Ghemawat 2003]
- At-least-once + dedup por chave (aplicação)
- Avro + Schema Registry → contrato borda↔consumidor

Rede oscilante e mensagens fora de ordem

Zonas de sombra (túneis, subsolos) entregam dados atrasados e fora de ordem. Solução em três camadas — o rigor de ordem que o Spanner obtém com TrueTime, o Flink emula com *watermarks*. [Corbett 2012; Carbone 2015]

BORDA

Compressão + buffer

Avro reduz o volume; *store-and-forward* guarda na desconexão e reenvia sem perda.

KAFKA

Ordenação por chave

Partição por `vehicle_id` mantém a ordem por veículo mesmo com produtores concorrentes.

FLINK

Event-time + watermarks

Processa pelo timestamp do veículo; *watermarks* reordenam pacotes atrasados. [Fragkoulis 2020]

RESULTADO

Ordenação estrita por veículo e timestamp — mesmo sob estresse de carga e conectividade intermitente.

Lambda vs. Kappa – e a decisão

CRITÉRIO	LAMBDA	KAPPA
Ideia	Camada batch (exata) + camada de velocidade + serving	Tudo é stream; reprocessa reproduzindo o log
Base de código	Dupla (mesma lógica em dois sistemas)	Única
Consistência	Risco de divergência entre camadas	Uma verdade, uma lógica
Manutenção	Alta	Menor
Reprocessar histórico	Camada batch nativa	Exige replay + Lakehouse para lote pesado

DECISÃO

O stream durável do Kafka unifica os caminhos e **elimina a dupla implementação** — adotamos **Kappa**, cobrindo o lote pesado com o Lakehouse.
[Fragkoulis 2020; Carbone 2015]

Kappa realizado por Lakehouse com Delta Lake

O object store é *key-value* sem atomicidade cross-key; o **log de transações do Delta** traz ACID serializável, *data skipping* e *time travel* ao data lake. [Armbrust 2020, 2021]



Três regimes para três SLAs distintos

BATCH / COLD PATH · HORAS

Apache Spark

RDD com tolerância por **linhagem**; reuso em memória até 20×. [Zaharia 2012]

- Consolida do Delta Bronze
- Manutenção preditiva; matriz de Markov
- Generaliza o **MapReduce** [Dean 2004]

STREAM · SEGUNDOS

Spark Streaming

Micro-batches (D-Streams) com **exactly-once** e saída idempotente. [Zaharia 2013]

- Hot path das emergências
- Orquestra a resposta na nuvem

STREAM · MILISSEGUNDOS

Apache Flink

Dataflow único; *ABS* (Chandy-Lamport) sem parar a execução. [Carbone 2015]

- Janelas + event-time + watermarks
- Rotas, congestão, score de condução

POR QUE SEPARAR

Vazão histórica (Spark batch) ≠ orquestração da emergência (Spark Streaming) ≠ latência mínima com estado (Flink). A reação sub-segundo à colisão é na borda; a nuvem orquestra a resposta.

Cada dado no banco certo para seu padrão de acesso

Raiz teórica: o **teorema CAP** — sob partição, consistência e disponibilidade não coexistem; segmenta-se o sistema e atribui-se a cada subserviço seu trade-off.

[Gilbert & Lynch 2002; Cattell 2011]

WIDE-COLUMN · AP

Cassandra

Telemetria. Escrita massiva (LSM), *clustering* por timestamp materializa a ordem. [Lakshman 2009; DeCandia 2007]

DOCUMENTO

MongoDB

Cadastrais e dossiês. Esquema flexível para sensores heterogêneos e relatórios.

KEY-VALUE · RAM

Redis

Cache quente. Estado ao vivo, cache de rotas, geoespacial (sub-ms).

RELACIONAL · CP

PostgreSQL / Delta

Fonte da verdade. Cobrança (ACID), DW/Gold, treino de ML.

MAPEAMENTO CAP

Telemetria (AP, *always-writeable*) vs. cobrança/dossiê (CP, consistência forte) — o mesmo trade-off do carrinho vs. checkout do Dynamo.

Painel de monitoramento em Streamlit

OLAP sobre a **Gold** (nunca sobre os OLTP [Chaudhuri & Dayal 1997]); ao vivo via **Redis** pub/sub. O cubo frota × tempo × pátio × empresa apoia a visão executiva [Gray 1997].

FROTA

Mapa ao vivo, veículos ativos, bateria média, alertas mecânicos.

RESERVAS & LOCAÇÕES

Funil de reservas e locações em andamento.

FINANCEIRO

Cobranças, receita e ajustes por locação.

EMERGÊNCIAS

Triagem que pisca em vermelho no payload crítico.

VIAGENS

Trajetos em curso, ETA, congestionamentos.

OFICINA

Manutenção preditiva e fila de higienização.

AO VIVO

Quase tempo real via Redis + auto-refresh.

HISTÓRICO

KPIs e tendências das marts Gold.

Cobrança automática e apoio a sinistros

COBRANÇA DINÂMICA · VALOR AO CLIENTE

Acionada na devolução

Valor final = base ± ajustes por km, tempo, consumo, infrações e score de condução (Flink).

Exactly-once idempotente evita cobrar em dobro [Zaharia 2013]; auditável por time travel.

↑ receita · ↓ contestações

EMERGÊNCIAS · HOT PATH

De colisão a dossiê de regulação

borda → alerts.critical → Spark → resposta

— Ação imediata: guincho, seguradora, substituto

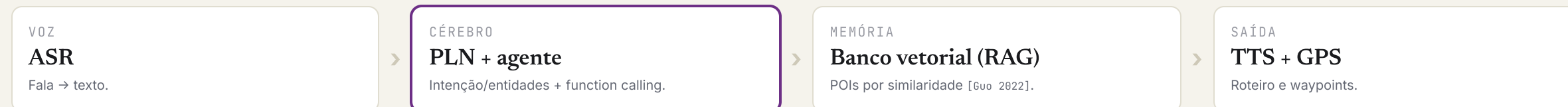
— Dossiê citável (sensores + vídeo + timeline)

↓ tempo de resposta e de regulação

Ponte entre as partes: a detecção (Parte A) alimenta a resposta e a triagem no dashboard (Parte B).

Concierge por voz: PLN + RAG (sem LLM paga)

O **RAG** ancora a geração em conhecimento recuperável (índice denso), reduz alucinação e dá **proveniência** auditável — a mesma base dos dossiês de emergência. [Lewis 2020]



EXEMPLO — "APÓS A REUNIÃO..."

Almoço típico bem avaliado + estacionamento fácil + exposição de arte nos gostos do cliente → o agente consulta localização, POIs (vetorial) e agenda, monta o roteiro e **injeta os waypoints no GPS**. Privacidade: consentimento, PII, *wake-word* on-device.

Três cenários: onde a modelagem cria valor

CRÍTICO · RESPOSTA RÁPIDA

Colisão em túnel

Detecção na borda (sub-segundo) → `alerts.critical` → Spark orquestra guincho, seguradora e substituto; o Flink reordena a telemetria que só sai do túnel depois.

Acende: **Fato_Sinistro**, `Dim_Sensor` (SCD-2), `Dim_TipoEvento`, **Dim_Tempo_Detalhe**.

↓ tempo de resposta

USUAL · FINANCEIRO

Devolução com condução agressiva

O Flink (warm path) computa o score → **Dim_FaixaConducao** "Agressivo"; na devolução, a cobrança *exactly-once* soma km, consumo e a infração.

Acende: **Fato_Cobranca**, `Fato_Telemetria_Diaria`, `Dim_FaixaConducao`, `Dim_TipoEvento`.

↑ receita / locação

ANALÍTICO · IA

Pico no aeroporto + pane + concierge

Markov condicional prevê a migração ao Galeão → reposiciona vazios; o batch prevê falha → agenda manutenção; o concierge (RAG) monta o roteiro.

Acende: `Fato_Movimentacao`, **Fato_Manutencao**, `Fato_Telemetria_Diaria`, `Dim_Patio`, `Dim_Tempo_Detalhe`.

↑ utilização · ↓ downtime · ↑ NPS

FUNDAMENTAÇÃO

Cada caso ativa fatos e dimensões específicos — e três deles **justificam a extensão do modelo** (`Dim_Tempo_Detalhe`, `Fato_Cobranca`, `Fato_Manutencao/Dim_Sensor`). A modelagem não é abstrata: aparece no crítico, no usual e no analítico.

Onde cada entidade modelada aparece

ENTIDADE + = EXTENSÃO	1 · COLISÃO CRÍTICO	2 · COBRANÇA USUAL	3 · FROTA ANALÍTICO
Fato_Sinistro +	dossiê + resposta	—	—
Fato_Cobranca +	recálculo da locação	valor ± acréscimos	—
Fato_Telemetria_Diaria +	buffer pré-impacto	km, velocidade, eventos	base do score / manutenção
Fato_Manutencao +	—	—	prob. de falha → agenda
Fato_Movimentacao	despacho ao pátio	—	Markov → reposicionamento
Fato_Reserva / Locação	veículo substituto	locação faturada	pool de disponíveis
Dim_Sensor + SCD-2	firmware na hora do sinistro	—	saúde do sensor
Dim_TipoEvento +	colisão	violação de trânsito	pane / superaquecimento
Dim_FaixaConducao +	—	Agressivo → tarifa	perfil por veículo
Dim_Tempo_Detalhe +	minuto 18:23	janelas da viagem	faixa horária / pico
Dim_Patio · Veículo · Cliente	pátio próximo · quem	retirada / devolução	origem / destino

Custo sob controle por design

BANDA / EGRESS

Filtrar na borda

Agregar e não enviar vídeo bruto corta o maior custo recorrente da frota móvel.

COMPUTE

Elasticidade

Autoscaling + instâncias spot/preemptíveis; clusters efêmeros no batch.

STORAGE

Tiering

Quente (Redis/Cassandra) vs. frio (Delta/object store), com TTL e Bronze→Gold.

PROVA DE CONCEITO EXECUTÁVEL

Sistema **docker-compose** (Kafka, Flink, Spark, Cassandra, Mongo, Redis, MinIO, Streamlit) validado; DDL do DW estendido testado em PostgreSQL. Free tiers: Dataproc/Flink Playground, Databricks, Atlas, Astra, Redis, DynamoDB.

Alternativas consideradas e descartadas

OPÇÃO DESCARTADA	POR QUE NÃO, NESTE CONTEXTO
Somente Data Warehouse	Batch e schema-on-write não dão tempo real nem absorvem sensores semiestruturados de alto volume.
Somente processamento batch	Emergências e roteirização exigem latência de segundos/milissegundos.
Arquitetura Lambda	Base de código dupla e risco de divergência; o problema é streaming-first.
Um único banco relacional	Não escala a escrita massiva de telemetria nem entrega cache sub-ms (CAP).
Stream de vídeo bruto pela rede	Banda móvel cara e finita; o vídeo fica na borda e sobe só o essencial.
Ingestão HTTP síncrona no banco	Sem contrapressão nem replay; a frota simultânea causaria perda de pacotes.

Fundamentos – referências primárias

Armbrust, M. et al. (2015). Spark SQL. SIGMOD.

Armbrust, M. et al. (2020). Delta Lake. PVLDB.

Armbrust, M. et al. (2021). Lakehouse. CIDR.

Carbone, P. et al. (2015). Apache Flink. IEEE Data Eng. Bull. + Lightweight Asynchronous Snapshots. arXiv:1506.08603.

Cattell, R. (2011). Scalable SQL and NoSQL Data Stores. SIGMOD Record.

Chang, F. et al. (2006). Bigtable. OSDI.

Chaudhuri, S. & Dayal, U. (1997). Overview of Data Warehousing and OLAP. SIGMOD Record.

Corbett, J. C. et al. (2012). Spanner. OSDI.

Dean, J. & Ghemawat, S. (2004). MapReduce. OSDI.

DeCandia, G. et al. (2007). Dynamo. SOSP.

Fragkoulis, M. et al. (2020). Survey on Stream Processing. arXiv:2008.00842.

Ghemawat, S. et al. (2003). The Google File System. SOSP.

Gilbert, S. & Lynch, N. (2002). Brewer's Conjecture (CAP). ACM SIGACT.

Gray, J. et al. (1997). Data Cube. DMKD.

Guo, R. et al. (2022). Manu: Cloud Native Vector DB. PVLDB.

Kreps, J. et al. (2011). Kafka. NetDB.

Lakshman, A. & Malik, P. (2009). Cassandra. LADIS/SIGOPS.

Lewis, P. et al. (2020). Retrieval-Augmented Generation. NeurIPS.

Zaharia, M. et al. (2012). Resilient Distributed Datasets (RDD). NSDI.

Zaharia, M. et al. (2013). Discretized Streams. SOSP.

Síntese: fundamentos → arquitetura → valor

- **Fundamentada** — cada camada ancorada em papers seminais (VLDB, DW, streaming, NoSQL).
- **Borda + Kafka + Kappa/Delta** — banda poupada, ordem por veículo, ACID lote↔stream.
- **Três regimes** — Spark (batch/emergências) e Flink (rotas).
- **Poliglota** — CAP aplicado: Cassandra, MongoDB, Redis, PostgreSQL/Delta.
- **DW estendido** — vira a camada Gold; novos fatos de telemetria/cobrança/sinistro.
- **Serving + IA** — dashboard, cobrança auditável, dossiês e concierge RAG.
- **Valor ao cliente** — ↑ utilização e receita, ↓ custo, downtime e tempo de resposta.
- **Executável** — sistema em docker-compose, validado.

CONCLUSÃO

Uma arquitetura **resiliente, escalável e em tempo real**, teoricamente fundamentada, que envolve o DW da Avaliação 02 e entrega **valor mensurável** ao consórcio com custo de nuvem sob controle.

Obrigado. Perguntas? Izabela Lima da Silva · Caio Meirelles · 2026.1